TÉCNICO LISBOA



Smart Models, Safer Protocols? Benchmarking LMs for Distributed Algorithm Verification

This is the Thesis Subtitle if Necessary

Luana Lourenço Ferraz

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Miguel Pupo Correia

Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Prof. Miguel Pupo Correia
Members of the Committee: Prof. Name of First Committee Member
Dr. Name of Second Committee Member
Eng. Name of Third Committee Member

Month 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetuer erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this work possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life.

To each and every one of you – Thank you.

Abstract

Nulla facilisi. In vel sem. Morbi id urna in diam dignissim feugiat. Proin molestie tortor eu velit. Aliquam erat volutpat. Nullam ultrices, diam tempus vulputate egestas, eros pede varius leo, sed imperdiet lectus est ornare odio. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Proin consectetuer velit in dui. Phasellus wisi purus, interdum vitae, rutrum accumsan, viverra in, velit. Sed enim risus, congue non, tristique in, commodo eu, metus. Aenean tortor mi, imperdiet id, gravida eu, posuere eu, felis. Mauris sollicitudin, turpis in hendrerit sodales, lectus ipsum pellentesque ligula, sit amet scelerisque urna nibh ut arcu. Aliquam in lacus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nulla placerat aliquam wisi. Mauris viverra odio. Quisque fermentum pulvinar odio. Proin posuere est vitae ligula. Etiam euismod. Cras a eros.

Keywords

Distributed Algorithms, Language Models, Program Verification, Benchmark.

Resumo

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Mauris placerat eleifend leo. Quisque sit amet est et sapien ullamcorper pharetra. Vestibulum erat wisi, condimentum sed, commodo vitae, ornare sit amet, wisi. Aenean fermentum, elit eget tincidunt condimentum, eros ipsum rutrum orci, sagittis tempus lacus enim ac dui. Donec non enim in turpis pulvinar facilisis. Ut felis. Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede.

Palavras Chave

Sistemas distribuidos; Modelos de Linguagem; Verificacao de Programas; Benchmark;

Contents

1 Introduction **1**

1.1 Work Objectives . . . 4

1.2 Expected Contributions . . . 5

1.3 Dissertation Outline . . . 5

1.3.1 Results of the Pseudocode-Level Error Tests . . . 6

2 Background and Related Work **7**

2.1 Distributed Algorithms . . . 8

2.2 Language Models . . . 11

2.2.1 Large Language Models (LLMs) . . . 11

2.2.2 Small Language Models (SLMs) . . . 12

2.2.3 Evaluating Reasoning Beyond Correctness . . . 12

2.3 Program Verification . . . 13

2.4 Benchmarking . . . 14

2.4.1 Benchmark Components . . . 15

2.4.2 Benchmark Properties . . . 15

2.4.3 Benchmarking LLMs . . . 16

2.5 Prompt Engineering . . . 17

3 Implementation **19**

3.1 System Architecture . . . 20

3.1.1 Algorithm and Prompt Repositories Component . . . 23

3.1.1.A Problem Selection . . . 23

3.1.1.B Prompt Design . . . 25

3.1.2 Model Interface component . . . 26

3.1.3 Evaluation Layer Component . . . 26

3.2 Summary . . . 27

3.3 next . . . 28

4 Evaluation **29**
4.1 Evaluation Procedure 30
4.2 Robustness Tests with Modified Algorithm Descriptions 31
4.2.1 Introduction of Larger Models and Extended Robustness Testing 32
4.2.2 Motivation for Pseudocode-Level Errors 32
5 This is the Fifth Chapter **33**
5.1 Maecenas vitae nulla consequat 34
5.2 Proin ornare dignissim lacus 35
6 Conclusion **37**
6.1 Conclusions 38
6.2 System Limitations and Future Work 39
Bibliography **39**
A Code of Project **41**
B A Large Table **47**

List of Figures

3.1 Data flow of the evaluation framework. 21

5.1 Test Environment 34
5.2 Adaptation System Behavior. a) Test 4. b) Test 5 36

List of Tables

2.1 Task-specific properties in distributed algorithms. 9

3.1 Implementation structure of the evaluation framework 23
3.2 Structure of the algorithm input template 24
3.3 Algorithms Included in the Evaluation 25
3.4 Classification rules used by the results analysis script 27

5.1 Network Link Conditioner Profiles 35

B.1 Example table 48
B.2 Example of a very long table spreading in several pages 48
B.3 Sample Table. 50

List of Algorithms

Listings

A.1 Example of a XML file. 41
A.2 Assembler Main Code. 42
A.3 HTML with CSS Code 43
A.4 HTML CSS Javascript Code 45
A.5 PYTHON Code 45

Acronyms

1

Introduction

Contents

1.1 Work Objectives 4

1.2 Expected Contributions 5

1.3 Dissertation Outline 5

Distributed algorithms are the foundation of modern computing, supporting critical infrastructures such as cloud platforms, databases, and globally coordinated systems. Ensuring the correctness of distributed algorithms is essential, since concurrency, asynchrony [**?**, **?**], and the inevitability of failures can lead to errors that while subtle may be severe. However, verifying these algorithms is a complex and time-consuming task, which motivates the use of automated tools, despite their limitations. Nowadays, distributed algorithms are verified through what can be called traditional verification approaches. These include theorem proving and model checking, which, while rigorous, present some limitations. First, they require significant expertise in formal logic and proof construction (in case of theorem proving) [**?**, **?**] and involve a steep learning curve, making them difficult to adopt in practice. Second, they suffer from the state-space explosion problem, where the number of states grows exponentially with the size of the problem [**?**]. Moreover, the verification process itself is often time-consuming, both in terms of constructing the formal specifications and executing the verification. Finally, these techniques suffer from scalability limitations, as each new algorithm or variation typically requires a new specification, and may quickly become infeasible for large families of algorithms to analyze, as system size grows.

These limitations motivate the exploration of alternative verification approaches that are more accessible and partially automated.

Large Language Models can help overcome some limitations of traditional verification approaches by making the process more accessible, flexible, and partially automated. By reasoning over natural language descriptions, pseudocode, and correctness properties, an LM can help identify relevant invariants, highlight suspicious execution scenarios, and generate candidate counterexamples without requiring a complete formal model from the beginning. This can reduce the initial verification effort and guide human experts toward the most relevant parts of the algorithm to analyze formally, having done some initial testing that can save a lot of time in formal verification.

However, LLMs do not provide the same guarantees as formal methods. They cannot exhaustively explore all possible states or produce fully reliable proofs. For this reason, their value is mainly as a complementary tool: they can help narrow down the verification problem, suggest useful directions, and make the process faster and easier before applying traditional verification techniques.

Given the recent success and widespread accessibility of LLMs, it is natural to explore whether they can be used to address some of these challenges. This raises an important question: *Can Language Models (LMs) support or complement the verification of distributed algorithms by reasoning over their descriptions and correctness properties?*

The verification process of distributed algorithms can be accelerated with the use of LMs, especially in first stages of verification where the goal is to understand the algorithm, reason about its behavior, and identify possible correctness issues. Instead of, completely replace traditional tools such as model checking or theorem proving, the LM can act as a verification assistant.

It can do tasks such as identifying candidate invariants, reasoning about safety and liveness properties, detecting inconsistencies or edge cases, suggesting possible counterexamples, and even helping translate informal algorithm descriptions into more structured specifications. It can also perform a simple analysis on whether the algorithm satisfies the properties it emplies to satisfy. Therefore, the LM can reduce part of the manual and repetitive effort required from experts, particularly before applying more rigorous formal verification tools, avoiding unnecessary iterations.

Correct verification of distributed algorithms supports the reliability of large-scale computing. A single undetected flaw in consensus, broadcast, or coordination mechanisms can cascade into system-wide failures. For example, Gabrielson describes an Amazon outage in which a single faulty server caused a site-wide failure due to error propagation and load-balancing interactions [**?**]. At the same time, advances in artificial intelligence, particularly LMs, a category in which we include LLMs, have demonstrated strong performance in reasoning [**?**], problem-solving, and knowledge recall. Exploring whether these models can serve as verification assistants is therefore both timely and potentially transformative.

Verification is fundamentally challenging because distributed systems must preserve safety and liveness properties [**?**] in environments that may involve asynchrony, unpredictable delays, and multiple types of failures (e.g., crash, omission, and Byzantine). Formal methods mentioned above, theorem proving and model checking provide, strong guarantees but require specialized expertise, extensive manual effort, and face obvious scalability limits [**?**]. On the other hand, LMs operate probabilistically, are sensitive to prompting strategies, and cannot guarantee correctness. This gap between formal rigor and probabilistic reasoning makes the problem non-trivial, underscoring the need of a formal, structured and rigorous evaluation framework.

Previous approaches to verification have relied primarily on formal methods and specialized tools, which, while successful [**?**], have not scaled well to increasingly complex distributed algorithms [**?**]. Meanwhile, the potential of LMs has only recently emerged, and their application to verification remains underexplored. Unlike model checkers, language models cannot exhaustively explore all states or produce proofs, but they may provide useful heuristics, partial verification, or counterexamples. This work lies in experimentally testing these models as alternative or complementary tools to traditional techniques.

This project proposes to systematically evaluate LMs on representative distributed algorithms [**?**, **?**]. The focus will be on their ability to identify invariants, reason about failures, and verify correctness. The benchmark will be designed to capture both strengths and limitations of these models, offering insights into whether they can contribute to automatic verification either as standalone tools or as assistants to human experts. While the probabilistic nature of LMs imposes clear limitations, the study aims to assess their practical utility and define scenarios where they can complement formal verification.

1.1 Work Objectives

The main objective of this work is to develop and apply a benchmarking framework capable of evaluating the ability of LMs to verify distributed algorithms through natural language understanding and reasoning. While traditional verification approaches, such as model checking, offer rigorous guarantees, they are computationally demanding and require significant formal expertise. For instance, theorem proving approaches have verified consensus protocols using Coq [**?**] with proof artifacts exceeding 50,000 lines [**?**, **?**]. In contrast, model checking approaches such as Spin [**?**] encounter state-space explosion when verifying Paxos [**?**], growing from thousands to tens of millions of states, and verification time increases from milliseconds to several minutes, making exhaustive verification infeasible beyond small configurations. In addition, both approaches require significant upfront effort to model systems and construct proofs prior to verification.

Based on these limitations, this work explores whether LMs can serve as a practical alternative or complementary tool for distributed algorithm verification. More specifically, the objectives of this work are:

- To develop a structured benchmark for evaluating the reasoning capabilities of LMs in the verification of distributed algorithms.
- To verify whether LMs can correctly reach a conclusion about the correctness properties of a given distributed algorithm, including both safety and liveness properties.
- To evaluate the ability of LMs to understand and reason under asynchronous system assumptions.
- To analyze how LMs perform under different failure models, including crash, omission, and Byzantine failures.
- To test if LMs can detect correctness violations in both correct and faulty variants of representative distributed algorithms.
- Even though the main goal is not to evaluate reosing but correct or incorrect conclusion without explanation, in some specific cases, reasioning will be addressed. In this case, evaluate LMs generation capabilities. If they are meaningful, logically grounded explanations or counterexamples when identifying potential violations.
- To compare different types of language models, ranging from large proprietary models to smaller open-source models, in order to identify their strengths and limitations.
- To investigate whether LMs can complement traditional verification methods by reducing manual effort, guiding formal verification, and supporting AI-assisted correctness reasoning.
- To establish a reusable evaluation framework that can support future research on the use of language models for distributed algorithm verification.

The proposed benchmark will combine a set of representative distributed algorithms, with both correct and faulty variants, and a standardized procedure to prompt and evaluate different models. By varying correctness properties, system assumptions, and failure models across tasks, the benchmark aims to reveal the strengths and limitations of LLMs and SLMs in replicating formal verification reasoning and detecting violations of distributed algorithm properties.

1.2 Expected Contributions

This work is expected to advance the research on the intersection of distributed systems, artificial intelligence, and program verification. The main contributions are:

- The development of a domain-specific benchmark that enables the systematic evaluation of LM reasoning in the verification of distributed algorithms. This framework will not only facilitate comparison between models but also establish methodological standards for assessing reasoning quality, correctness detection, and fault-tolerance awareness.

- The project will contribute new insights into how LMs interpret algorithmic structure and system properties, clarifying their limitations and potential advantages relative to formal verification tools.

- By ensuring that the benchmark is extensible and reproducible, the results of this work are expected to provide a foundation for subsequent studies and practical applications in AI-assisted verification.

- This work also aims to demonstrate that LMs can become a valuable complementary resource in verifying distributed algorithms, paving the way for more accessible and efficient verification methodologies.

1.3 Dissertation Outline

The remainder of the document is structured as follows: Chapter 2 presents background about the distributed algorithms, language models, program verification, benchmarking and prompt engineering. Chapter 3 explain the benchmark structure and implementation, from the algorithms templates, to the prompt engineering techniques used and language models querying. It also includes the results collecting and interations. Chapter 4 presents an evaluation methodology to assess the benchmark. The results from the experimental evaluation are presented and discussed. Next, we compare the different models and prompts, providing a discussion on its similarities and differences. Finally, Chapter 5 concludes this dissertation, fostering future works.

1.3.1 Results of the Pseudocode-Level Error Tests

The pseudocode-error experiments produced what is arguably the most revealing finding of the entire evaluation. Three variants of the Byzantine reliable broadcast algorithm were created with genuine logical errors introduced directly into the protocol pseudocode: in the first variant (-v), the threshold for triggering support messages was raised from 1 to f+1 received type0 messages, which prevents any correct process from initiating the support chain when a single correct broadcaster sends the initial message, thereby violating RB-Validity; in the second variant (-i), the delivery threshold was reduced from f+1 to 1 type1 message, which allows a single Byzantine process to forge a support message and force delivery of a message that was never broadcast, thereby violating RB-Integrity; and in the third variant (-a), the forwarding rule was removed entirely from the type1 message handler, meaning that a process that delivers does not propagate the support to other correct processes, thereby violating RB-Agreement.

These three variants were tested in versions 41–43 using DeepSeek-Chat (V3), LLaMA-3.3-70B-Groq, and Gemini-2.5-Flash-Lite, with the same four prompt configurations used throughout the evaluation (chain-of-thought, role-based, strict-format, and zero-shot). The original correct algorithm was included in version 43 as a control, and both DeepSeek-Chat and LLaMA-3.3-70B-Groq achieved 100

However, when presented with the broken variants, the results were fundamentally different: across a total of 96 valid test cases (excluding timeouts and rate-limit errors), only one single instance of error detection occurred. DeepSeek-Chat marked all three properties as satisfied in every one of its 36 valid tests across the three broken variants, yielding a 0

These results demonstrate a fundamental asymmetry in the verification capabilities of current language models: while all tested models achieved near-perfect accuracy when confirming that a correct algorithm satisfies its stated properties (consistently above 95

This finding has significant implications for the use of language models in formal verification contexts: these models can serve as effective tools for confirming or documenting the correctness of algorithms that are already known to be correct, but they cannot currently be relied upon to independently discover violations or errors in protocol logic. The distinction between verification (checking a proof) and falsification (finding a counterexample) represents a boundary that the current generation of models, regardless of parameter count, does not reliably cross.

2

Background and Related Work

Contents

2.1 Distributed Algorithms 8
2.2 Language Models 11
2.3 Program Verification 13
2.4 Benchmarking 14
2.5 Prompt Engineering 17

In this section, we introduce the key concepts and related work that support the development of this project. We begin by presenting distributed algorithms, both at a conceptual level and through representative abstractions and concrete examples. We then discuss language models, outlining their evolution and highlighting the most recent and advanced systems relevant to our study. The section continues with an overview of program verification, a core component of our work, examining existing approaches and motivating the need for more accessible and efficient alternatives.

Because our solution requires a systematic methodology to evaluate language models, we next explore benchmarking techniques and how they can be used to assess LM performance in distributed algorithm verification in a formal and rigorous manner. Finally, we survey prompt engineering methods, which provide effective strategies for guiding language models to produce accurate and reliable reasoning.

2.1 Distributed Algorithms

Distributed algorithms are designed to run on multiple interconnected computing entities, commonly referred to as nodes or processes, within a distributed system. They provide the core mechanisms that enable these independent components to cooperate, coordinate, and reach agreement despite the challenges introduced by concurrency, asynchrony, and the possibility of failures. As defined by Lynch in her classical book [**?**], a distributed algorithm specifies the behavior of a set of processes that interact through some sort of communication, such as message passing over potentially unreliable networks or shared memory. These algorithms govern critical coordination tasks including consensus, broadcast, leader election, and mutual exclusion. Although their implementation differs depending on whether communication is based on message passing or shared memory, both models share the fundamental challenge that no single process has a complete and instantaneous view of the global system state, due either to communication delays and faults in message passing, or to concurrent access and interleaving in shared memory.

A central aspect of distributed computation lies in the system model under which the algorithm operates. In a synchronous system, message transmission and process execution occur within known and bounded time limits [**?**]. This predictability simplifies reasoning and enables deterministic progress guarantees. In contrast, asynchronous systems impose no timing assumptions: messages can be delayed arbitrarily, and process speeds may vary without bound. Fischer, Lynch, and Paterson proved that deterministic consensus is impossible in purely asynchronous systems prone to even a single crash failure, known as the FLP impossibility result [**?**]. This distinction between synchrony models is fundamental, as it shapes both the design and the verifiability of distributed protocols.

Another key dimension is the fault model, which defines how processes and communication links may deviate from their expected behavior. The crash-failure model assumes that processes may halt unexpectedly and never recover. In the omission model, processes or links may occasionally fail to deliver messages.

The most general and adversarial case, the Byzantine model, allows arbitrary and potentially malicious behavior, including sending inconsistent messages to different peers [**?**]. Each fault model imposes different requirements on correctness and fault tolerance, typically parameterized by the number of tolerated faults F in a system of N processes.

Distributed algorithms are defined according to correctness properties, broadly categorized into safety and liveness [**?**]. A safety property asserts that "nothing bad ever happens" (e.g., no two processes decide different values), while a liveness property ensures that "something good eventually happens" (e.g., all correct processes eventually decide). In practice, these general properties are expressed through task-specific correctness conditions, such as agreement, validity, or termination, which refine what safety and liveness mean for a particular algorithm or abstraction. The most common properties are summarized in Table 2.1.

Table 2.1: Task-specific properties in distributed algorithms.

Property	**Type**	**Definition**
Validity	Safety	If a correct process decides (or delivers) v, then v was proposed by some process or v = $\perp$. [**?**]
Agreement	Safety	No two correct processes decide (or deliver) differently. [**?**]
Integrity	Safety	For any message M, every correct process p delivers M at most once, and if sender(M) is correct then M was previously broadcast by sender(M). [**?**]
Total Order	Safety	If two correct processes deliver two messages M1 and M2 then both processes deliver the two messages in the same order. [**?**]
Consistency	Safety	all correct processes end in states that do not contradict each other; the system behaves as if operations occurred in a valid order. [**?**]
No Duplication	Safety	Derives from Integrity. A message is never delivered more than once. [**?**]
No Creation	Safety	Derives from Validity. No message is delivered unless it was actually sent. [**?**]
Termination	Liveness	Every correct process eventually decides (or delivers). [**?**]
Vector Validity	Safety	Every correct process that decides, decides on a vector V of size n: (i) For all p, if pi is correct, then either V[i] is the value proposed by pi or $\perp$; and (ii) at least (f +1) elements of V were proposed by correct processes. [**?**]

The benchmark proposed in this work considers a representative subset of these properties across a family of distributed algorithms chosen for their clarity and well-understood correctness guarantees [**?**]. Rather than focusing on the full complexity of large-scale production protocols, the benchmark prioritizes simpler yet expressive abstractions, such as Binary Consensus, Multi-Value Consensus, Reliable Broadcast, and their fault-tolerant or terminating variants, that expose core safety and liveness reasoning patterns while remaining accessible to language model analysis. These abstractions correspond to the classical building blocks of distributed computing as formalized in classical books by Lynch [**?**] and Raynal [**?**]. All

selected algorithms are defined in the message-passing model, assuming that processes communicate exclusively through message exchange.

Reliable Broadcast is a foundational communication abstraction that ensures all correct processes deliver the same set of messages even in the presence of failures. Hadzilacos and Toueg [**?**] introduced a layered perspective that decomposes broadcast abstractions, reliable, uniform, and atomic, into modular components with well-defined correctness properties. This decomposition clarifies the conceptual relationships between different broadcast semantics and illustrates how complex protocols can be derived compositionally from simpler ones, a perspective developed extensively in foundational treatments such as Raynal [**?**].

Similarly, consensus abstractions, including binary, multi-value, and vector-based consensus, capture the fundamental task of reaching agreement under various synchrony and fault assumptions. Their definitions, behavior, and correctness criteria are formalized in classical references such as Lynch [**?**] and Raynal [**?**]. These abstractions map directly onto the task-specific safety and liveness conditions listed in Table 2.1, making them well suited for evaluating whether language models can identify property satisfaction or violation.

Recent research continues to expand the landscape of distributed algorithms, not only through manual design but also through automated synthesis techniques. An example is the work of Vaz et al. [**?**], which presents the first use of reinforcement learning to automatically generate fault-tolerant Reliable Broadcast algorithms. Their approach integrates an RL-based generator with a model-checking oracle to iteratively synthesize correct and efficient protocols. This demonstrates that the space of distributed algorithms is still evolving, with new designs emerging from both human insight and automated methods. In light of this, the present work includes one such automatically synthesized Reliable Broadcast algorithm as part of the benchmark, allowing us to evaluate whether language models can reason about correctness even for recently proposed, machine-generated protocols that have not yet become canonical.

Verifying the correctness of distributed algorithms remains a formidable challenge. Their behavior depends not only on local computation but also on the global interleaving of events, message timings, and failure scenarios. Formal verification techniques such as model checking and theorem proving [**?**] have been extensively applied to this domain but face scalability and automation limitations due to the exponential growth of system states. Additionally, traditional model checking requires engineers to manually express correctness properties in temporal logic and encode the system as a state machine—an additional effort that introduces complexity and raises the entry barrier for practical use, especially for those unfamiliar with formal specification languages [**?**]. These challenges motivate the exploration of alternative, data-driven approaches, such as leveraging LMs, to reason about distributed behavior through structured natural language understanding and probabilistic inference.

2.2 Language Models

Language Models (LMs) are statistical systems trained to predict and generate sequences of text. Modern approaches build upon deep neural architectures such as the Transformer [**?**], enabling models to learn long-range dependencies and contextual relationships within massive datasets. Through large-scale pretraining on natural language (e.g., Wikipedia articles or news), code (e.g., Python or Java), and formal text (e.g., TLA+ or Z notation), such as logical or specification syntax, these models acquire good generalization abilities that allow them to perform diverse reasoning and generation tasks. Depending on their scale, architecture, and training data, language models can be categorized as Large Language Models (LLMs) or Small Language Models (SLMs). While LLMs typically rely on hundreds of billions of parameters and require substantial computational infrastructure, SLMs operate at a much smaller scale, commonly between 1M and 10B parameters [**?**, **?**]. This drastic reduction in size fundamentally shapes their strengths and limitations.

Recent progress in large language models has demonstrated that such systems can perform tasks involving structured reasoning, code synthesis, and logic-based problem solving [**?**, **?**]. This opens the possibility of using them as assistive tools for the work of verifying distributed algorithms. Unlike traditional verification methods that rely on exhaustive exploration or formal proofs, LMs operate through probabilistic reasoning and pattern recognition, which might allow them to generalize across algorithmic families and failure scenarios. However, there are some limitations that pose significant challenges, such as sensitivity to prompt phrasing, lack of formal guarantees, reasoning inconsistencies and hallucinations that are syntactically plausible outputs that in fact are incorrect [**?**]. A well-designed benchmark is therefore essential to assess LMs in a systematic, controlled, and reproducible manner, ensuring that models are evaluated appropriately according not only to its correctness and reasoning quality, interpretability and robustness but also that, if any weaknesses emerge, they are clearly identified and interpreted in context.

2.2.1 Large Language Models (LLMs)

LLMs, such as GPT-4 [**?**] or Perplexity AI [**?**], contain hundreds of billions of parameters and exhibit remarkable performance in natural language understanding, reasoning, and code generation. Their extensive pretraining across heterogeneous datasets allows them to internalize diverse algorithmic and logical structures, making them particularly suitable for tasks that require understanding system assumptions, analyzing code, and evaluating correctness properties. However, recent works have questioned whether these models genuinely "reason" or merely reproduce statistical patterns, a discussion known as the *illusion of thinking* [**?**, **?**]. This debate has direct implications for verification-oriented tasks, where genuine logical reasoning is required. For instance, Shojaee et al. [**?**] show that LLMs may appear to "think less" as task complexity increases, while Lawsen [**?**] argues that such failures may come from task design or

token-length limitations rather than lack of reasoning ability. These insights highlight the need for carefully designed, unambiguous prompts in any verification benchmark. Prompt engineering, task solvability, and reasoning transparency become central to fair evaluation.

2.2.2 Small Language Models (SLMs)

Small Language Models (SLMs) are transformer-based models with significantly fewer parameters, typically between 1M and 10B, designed for efficient, low-latency inference on consumer-grade hardware [**?**, **?**]. Representative examples include Yi-Coder (1.5B) and (9B) [**?**] and Phi-2 (2.7B) [**?**]. In contrast to LLMs, which require large-scale computational resources and excel at broad generalization, SLMs prioritize efficiency, interpretability, and deployability, making them suitable for on-device or resource-constrained environments [**?**].

Recent work shows that SLMs can still exhibit non-trivial reasoning abilities when augmented with targeted training strategies. Studies on reasoning distillation demonstrate that small models can inherit structured reasoning patterns from larger models, improving performance on logic-oriented tasks [**?**]. Likewise, empirical evaluations of SLMs on code generation [**?**] show that models below 7B parameters can perform competitively on standard benchmarks, capturing syntactic regularities and control-flow patterns relevant to program understanding.

However, their smaller parameter count can limit the ability to perform complex multi-step reasoning, which is central to the verification of distributed algorithms. Compared to larger models, SLMs may struggle with reasoning chains involving multiple interacting conditions, such as invariants, message ordering, and failure scenarios. Verification tasks often require nested conditional reasoning (e.g., reasoning about different failure cases or execution paths), which can be challenging for smaller models due to limited representational capacity. For this reason, SLMs are included in this work to evaluate how model scale impacts the capacity to reason about distributed algorithm correctness.

2.2.3 Evaluating Reasoning Beyond Correctness

Recent studies on LM evaluation [**?**, **?**] highlight a key insight for this work: assessing only the final correctness of a model's output (e.g., whether it answers "true" or "false" to a verification query) is often insufficient to fully capture its reasoning capabilities. Language models may fail not due to a lack of reasoning ability, but because the evaluation setup overlooks important aspects of how and why the answer was produced.

In the context of this benchmark, several evaluation strategies are being considered. One possibility is to focus strictly on binary outcomes, whether the model correctly determines if a given distributed algorithm satisfies a target property. This approach simplifies scoring and supports automation at scale. Alternatively, the benchmark may also consider the reasoning trace provided by the model, its justification or explanation

for the verdict. This would allow for a more nuanced understanding of the model's decision process and its alignment with formal reasoning principles.

A third, intermediate strategy is collecting both the binary verdict and the associated reasoning, while formally evaluating only the former. In this setup, the reasoning output is stored systematically and may be used later for manual review, qualitative analysis, or to guide improvements in prompt design and model comparison. This hybrid approach balances automation with interpretability, and leaves room for future iterations of the benchmark to incorporate structured evaluation of reasoning quality.

At this stage, no final decision has been made regarding which evaluation strategy will be adopted. The benchmark is being designed to remain flexible and to accommodate all three alternatives, with a particular emphasis on supporting reproducibility, future manual inspection, and iterative refinement of evaluation criteria.

Finally, work on evaluation of code generation [**?**] emphasize that simply running unit tests is not sufficient for assessing code understanding or reasoning. In a similar spirit, the proposed benchmark focuses on domain, specific properties, such as safety, liveness, and fault tolerance, bridging the gap between generic LM evaluation and the verification of distributed systems.

2.3 Program Verification

Program verification is the study of proving that a program conforms to a formal specification. It relies on mathematical reasoning to demonstrate that every execution of a program satisfies the properties defined in its specification. This typically involves modeling program behavior as a transition system, expressing correctness conditions in a logical formalism, and establishing invariants or proof obligations that exclude incorrect executions and guarantee eventual progress [**?**, **?**]. In distributed systems, the verification effort is especially challenging because message delays, asynchrony, concurrency, and failures lead to large and complex execution spaces. These difficulties motivate the exploration of language models as tools for analyzing and explaining correctness in distributed algorithms.

Program verification has two major classes of formal methods: theorem proving and model checking. Together, they provide foundational techniques for establishing that a program conforms to its formal specification [**?**, **?**].

Theorem proving is based on deductive reasoning. Proofs can be constructed manually or with the assistance of proof systems such as Isabelle [**?**]. As surveyed by Nawaz et al. [**?**], theorem provers are particularly effective for reasoning about infinite-state systems and highly abstract properties, since they allow symbolic manipulation of invariants and logical statements. This expressive power makes them suitable for verifying distributed algorithms whose behavior cannot be represented as a finite-state model. However, theorem proving generally requires human guidance, familiarity with formal logic, and careful construction

of proofs, which limits its degree of automation and its accessibility to non-experts.

Model checking offers a more automated alternative. It works by constructing a formal model of the system, typically a finite-state transition system, and then exhaustively exploring all states that the system can reach from its initial configuration. During this exploration, the model checker verifies whether each reachable state satisfies the properties specified in a temporal-logic formula. If a property is violated, the tool produces a counterexample in the form of an execution trace that shows exactly how the system can reach an incorrect state. Tools such as SPIN [**?**] and UPPAAL [**?**] are widely used to verify communication protocols, control systems, and distributed algorithms. Model checking detection of violations and production of counterexamples are valuable for diagnosing design flaws. However, it is constrained by the state-space explosion problem, where the number of possible states grows rapidly with system complexity. This limitation can make model checking infeasible for systems with large or parameterized state spaces.

These difficulties motivate the exploration of complementary approaches to analyzing and explaining correctness in distributed algorithms. The aim of this work is to evaluate the ability of LMs to determine whether distributed algorithms satisfy their correctness properties and, when they do not, produce counterexamples that explain the violation. Program verification provides the formal basis for this evaluation, as the goal is to assess whether LLMs can approximate, at least partially, the results that formal verification traditionally produce. It defines the correctness criteria, such as safety and liveness, that specify how algorithms should behave under all admissible executions. These are reflected in concrete properties, such as validity, agreement, and termination, that apply under particular system and fault assumptions. Traditional verification methods, although rigorous and precise, often require specialized expertise, involve detailed formal modeling that can be difficult to apply in practice and are highly time consuming. In many cases, the state space grows so large that available computational resources, such as memory, are insufficient to complete the verification [**?**]. This work investigates whether LMs, through natural language reasoning and contextual understanding, can support or partially automate aspects of program verification in distributed settings.

2.4 Benchmarking

Benchmarking is a systematic process for evaluating and comparing the performance of systems, algorithms, or models under controlled and repeatable conditions. According to Nunes et al. [**?**], a benchmark provides a quantitative and qualitative basis for performance assessment through standardized workloads, metrics, and procedures. The fundamental goal of benchmarking is not only to measure raw performance, but to establish reproducibility, comparability, and insight into the underlying behaviour of the systems being evaluated.

Benchmarks have long been used across computing domains, from numerical simulation [**?**] and databases

[**?**] to distributed [**?**] and AI systems [**?**], as a means to establish reference performance indicators. As Nunes et al. emphasize, benchmarking should not be understood merely as running tests, but rather as an experiment guided by scientific principles: the benchmark should be reproducible, representative of realistic workloads, and grounded in well-defined metrics and methodologies. A well-designed benchmark allows different tools or models to be evaluated under equivalent conditions, exposing their strengths, weaknesses, and scalability characteristics.

2.4.1 Benchmark Components

The most common method to assess and compare the performance of alternative tools is to execute them with a set of representative test cases and compare results. Following the definition by Vieira et al. [**?**] and the methodology discussed by Nunes et al. [**?**], a benchmark typically includes three main components:

1. **Workload:** A set of representative test cases or input tasks designed to exercise the tools or models under evaluation.

2. **Metrics:** Quantitative and/or qualitative indicators that reflect how well the models fit their intended purpose.

3. **Procedures and Rules:** The experimental methodology that defines how the benchmark is executed, ensuring fairness and reproducibility.

These principles ensure that benchmarking results remain meaningful and transferable across studies. Together, these components define how the benchmark is structured and executed, ensuring that the evaluation process remains systematic, fair, and reproducible.

2.4.2 Benchmark Properties

Complementing these structural aspects, the benchmark must also define the correctness properties that characterize the expected behavior of the distributed algorithms under evaluation. According to Vieira et al. [**?**], a suitable workload should satisfy the following properties:

- **Representativeness:** Test cases should reflect realistic and typical scenarios of the domain where the benchmark will be applied.

- **Comprehensiveness:** The workload should cover all relevant aspects of the system's functionality.

- **Focus:** The workload should emphasize the critical features of the domain, balancing coverage and relevance.

- **Configurability:** It should be possible to customize the workload according to different experimental requirements.
- **Scalability:** The benchmark must scale in size and complexity while maintaining its representativeness.

As highlighted by Nunes et al. [**?**], these principles ensure that benchmarking results remain meaningful and transferable across studies. Furthermore, recent works such as Ye et al. [**?**] and Gupta et al. [**?**] emphasize the increasing role of benchmarks for evaluating *language models* and distributed systems, where reasoning accuracy and robustness are equally critical as performance. Ye et al. [**?**] evaluate LLMs through uncertainty quantification, showing the need for better metrics beyond accuracy, this aligns with our aim of assessing deeper reasoning abilities. IMSuite [**?**] is a benchmark designed to simulate distributed algorithms under different execution environments in order to measure characteristics related to its performance such as latency, message overhead, and scalability. It provides implementations of a variety of classical distributed algorithms and evaluates them through simulation, allowing researchers to compare their operational behavior across configurations and workloads. Although IMSuite also relies on distributed algorithms as its workload, it is only relevant mainly as an example of how distributed algorithms can be used to structure benchmark workloads. None of these benchmarks, or any existing one, address LM reasoning about correctness of distributed algorithms, which is the goal of our proposed benchmark.

2.4.3 Benchmarking LLMs

The drawbacks of traditional verification tools and the continuous progress of language models point toward a common direction: LLM-assisted verification. Recent work by Zheng et al. [**?**] introduces the use of a large language model as a judge, known as LLM-as-a-judge, where an LLM is responsible for evaluating the responses of other models on open-ended questions. Given two answers to the same prompt, the judge LLM selects the better response or declares a tie. In their experiments, GPT-4 achieved the best judging performance. Notably, the judge model was able to detect subtle failure modes such as verbosity bias, an attack in which a model repeats the same content using different phrasing to appear more thorough. The judge identified that one model's answer duplicated content, and correctly preferred the more concise and semantically coherent response.

Li et al. [**?**] show that LLMs can reliably act as evaluators rather than mere generators, assessing correctness, coherence, and reasoning quality in other models' outputs. These results indicate that LLMs can reliably evaluate the reasoning of other models, effectively substituting a human judge in complex comparative tasks.

This capability strengthens the motivation for our work, but it does not imply that judging open-ended answers and verifying distributed algorithms have the same level of complexity. In LLM-as-a-judge settings,

the model typically compares or evaluates textual responses according to criteria such as coherence, completeness, and reasoning quality. In contrast, distributed algorithm verification requires reasoning about all admissible executions under specific system and failure assumptions, including concurrency, asynchrony, and possible faults. Therefore, although previous results suggest that LLMs can detect subtle flaws and discriminate between correct and incorrect reasoning, it remains an open question whether they can perform the more demanding task of assessing whether distributed algorithms satisfy correctness properties.

Rather than assuming that LLMs can replace formal verification, this work investigates whether they can support it as intelligent assistants capable of hypothesis generation, early validation, error localization, or counterexample suggestion. This hybrid perspective, combining formal rigor with probabilistic reasoning, motivates the benchmark developed in this work, which evaluates how effectively LMs can reason about correctness in distributed algorithms and where their limitations become evident.

However, benchmarking LLMs requires a rigorous structure, as unreliable evaluation methodologies can easily lead to misleading conclusions. Recent work has highlighted the growing importance and difficulty of reliably evaluating language models, particularly as they are increasingly used not only as generators but also as evaluators. Zhou et al. [**?**] warn that benchmark contamination and test leakage can lead to misleading conclusions, emphasizing the need for carefully designed evaluation methodologies. In the same direction, White et al. [**?**] propose LiveBench, a continuously updated and contamination-resistant benchmark, showing that robust evaluation requires both freshness of tasks and rigorous experimental control. Focusing specifically on evaluation roles, Tan et al. [**?**] introduce JudgeBench, a benchmark for systematically assessing the reliability of LLM-based judges, demonstrating that judging itself is a non-trivial capability that must be explicitly measured. Together, these works establish that when LLMs are used as evaluators, benchmark design, robustness, and judge reliability become central concerns.

Ultimately, this project builds upon this convergence between formal verification and machine learning. By developing a structured benchmark for evaluating LMs in distributed algorithm verification, the work aims to test whether such models can correctly assess algorithmic properties such as termination, validity, and agreement, understand system assumptions (synchrony, failure thresholds), and generate meaningful, logically grounded explanations or counterexamples. Through this, we aim to explore how language models could complement or enhance existing verification methods, providing a scalable and flexible path toward AI-assisted correctness reasoning.

2.5 Prompt Engineering

Prompt engineering is the methodical practice of crafting text prompts to reliably guide LMs toward producing accurate, relevant, and interpretable outputs. As LMs have become powerful tools for reasoning, coding, and problem-solving, the formulation of prompts has emerged as a crucial interface between human

intention and model capability. The phrasing, structure, and contextual framing of a prompt can drastically affect the model's reasoning path, as well as the factual consistency and reliability of its response.

Early work on prompting showed that even a simple natural-language cue, such as the instruction "Let's think step by step", can substantially improve a model's reasoning performance, revealing that minimal guidance can elicit structured intermediate steps in zero-shot settings [**?**]. There are 3 main concepts of prompting:

- **Zero-shot** prompting introduced the idea that LMs can generalize to new tasks without fine-tuning, relying solely on pre-trained knowledge [**?**].
- **Few-shot** prompting, extended zero-shot later, where a small number of input–output examples embedded directly into the prompt allow the model to infer task structure from context [**?**].
- **Chain-of-thought** prompting formalized the strategy of encouraging models to articulate intermediate reasoning steps, further improving accuracy on reasoning-intensive tasks [**?**].

Subsequent research explored techniques that further structure the interaction between human and model. Role prompting defines an explicit identity or expertise for the model (e.g., "You are an expert in distributed algorithms and formal verification"), which has been shown to anchor responses to specific domains and improve factual consistency. Output-constrained prompting defines explicit answer structures (e.g., stepwise reasoning, tables, or classifications), improving interpretability and automating evaluation. These principles have been widely discussed in both academic and practical guidelines, including OpenAI's Best Practices for Prompt Engineering [**?**], and were further explored by Zhou et al. [**?**], who showed that advanced models can autonomously craft prompts that rival human-designed ones.

In general, this body of work highlights that prompt engineering is both an empirical science and a methodological discipline, since its effectiveness depends not only on creativity but also on systematic control of variables such as context, instruction framing, and output format. For research involving reasoning tasks, prompts act as experimental instruments: they define how models interpret the problem and what kind of reasoning they are expected to exhibit.

In the context of this work, these findings provide essential groundwork for developing a fair and reproducible benchmarking framework for language models in the domain of distributed algorithm verification. While various prompting strategies exist, this work focuses primarily on instruction-based prompting, which defines clear objectives and evaluation criteria and structured output prompting, ensuring consistent and easy comparisons across models. Chain-of-thought prompting is acknowledged as an important reasoning strategy but will be used only selectively, such as when generating counterexamples or reasoning traces as the reasoning is not the main focus of this project, but understanding the ability of algorithm verification. The detailed prompt templates and evaluation methodology will be introduced later in this document.

3

Implementation

Contents

3.1 System Architecture 20

3.2 Summary 27

3.3 next 28

3.1 System Architecture

This section presents the architecture of the implemented evaluation framework, designed to understand the ability of language models to verify distributed algorithms under different system assumptions. This framework combines a repository of distributed algorithms, a set of prompt templates based on different prompt engineering strategies, and a model interface responsible for querying the selected models with all combinations of algorithms and prompts. The generated outputs are then analyzed and compared according to the correctness properties expected for each algorithm.

The architecture is designed to test the same set of algorithms across different models and prompt types in a consistent way. Each algorithm is paired with each prompt template to create a final model query, which is submitted to each selected model, and each corresponding output is stored for later evaluation. This structure enables us to compare not only the performance of different language models and the knowledge and reasoning across different algorithms but also the impact of prompt engineering on distributed algorithm verification.

The framework is composed of four main components: *the algorithm repository*, *the prompt template repository*, *the model interface*, and *the response storage* and two execution scripts, one for the model interface and another for results storaging.

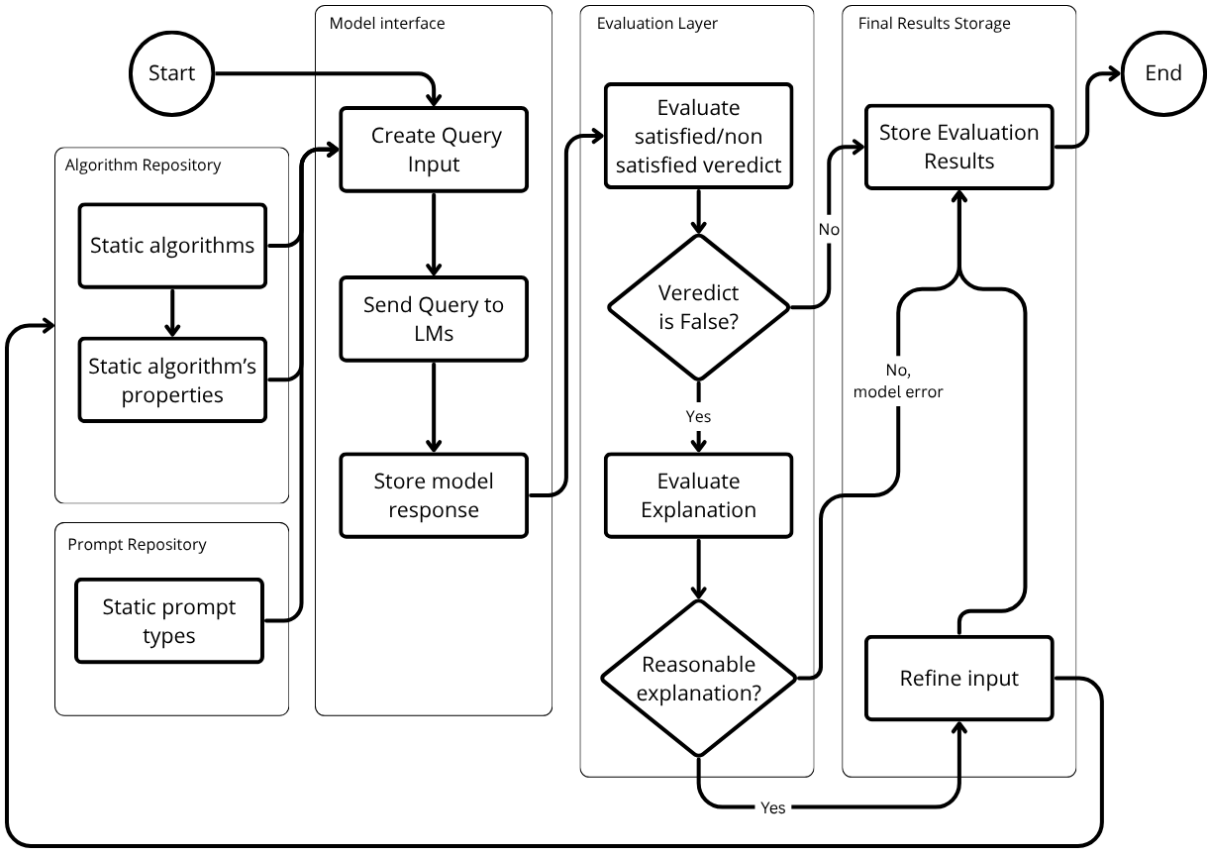


Figure 3.1: Data flow of the evaluation framework.

Figure 3.1 presents the data flow of the evaluation framework. The first elements of the system are the algorithm repository, `algorithms/` directory, which stores the distributed algorithms used in the evaluation, the `properties/` directory which stores only the correctness properties to be verified for each algorithm, kept in a separate directory to make them explicit in the prompt input, instead of requiring the model to extract them from the algorithm description itself, and the `prompts/` directory stores the static prompt templates used in the experiments, containing different prompt formulations, each following a specific prompt engineering strategy. The prompt templates are later combined with the algorithms to create the final prompts submitted to the models, being the model interface responsible for this using a python script that generates all combinations of the set algorithms and their corresponding properties and prompt types, starting the data flow when triggered. This design ensures that all models are evaluated under the same conditions and that differences in the outputs can be attributed to the model or prompt strategy rather than to inconsistencies in the input construction. Each generated prompt is then submitted to each selected model through the model interface. Therefore, the execution covers all combinations of algorithms, prompt strategies, and models.

Each response is then saved in the `results/` directory, following a structure organized by algorithm

and prompt type. For example, each iteration of the experiment has its results folder, being differenciated with the number of iteration. Inside an interation results folder, each algorithm has its own folder, and inside it the responses are grouped according to the prompt strategy used, and inside the prompt folder there is a file with the result, being its name the model used for this prompt combination. This leads to a very large number of output files due do the density of the experiment.

The second script is responsible for processing the stored model responses and rebuilding the Excel results file. This script scans all result folders, and treats the folder suffix as the version identifier. Storing the responses separately makes it possible to inspect each result manually when needed, repeat parts of the evaluation, and compare outputs across models and prompt types without rerunning the entire experiment. For each algorithm, it reads the corresponding file in the `properties/` directory to obtain the list of correctness properties that must be verified, this separation is important because the expected properties are not taken from the model output, they are explicitly defined before evaluation.

For each stored response, the script parses the model answer and compares it with the expected properties, supporting both plain text responses and JSON-stream responses produced by some model interfaces. Empty files, rate-limit errors, timeouts, connection errors, and unusable JSON-stream responses are classified as execution errors, and responses that do not mention enough expected properties, use wrong property names, or do not follow a recognizable verdict format are classified as incorrect output format. When the response follows the expected structure, the script checks each property individually. A property is counted as satisfied when the answer contains a recognized positive verdict, such as `[satisfied]`, `satisfied`, `true`, or `yes` and is counted as non-satisfied when the answer contains a recognized negative verdict, such as `[non satisfied]`, `not satisfied`, `false`, `violated`, or `not met`. If all expected properties are marked as satisfied, the result is classified as correct, on the other hand, if the response is structured but at least one property is marked as non-satisfied, the result is classified as wrong and the results are written into an Excel workbook. For each experimental version, it creates a separate sheet containing the results organized by algorithm, prompt type, and model, using a color-coded layout: green for correct answers, red for structured but wrong answers, orange for incorrect output format, and gray for execution errors. In addition, the script creates a global comparison sheet that aggregates results across all versions, including success rates by version, model rankings, prompt rankings, algorithm difficulty rankings, heatmaps for model–prompt, model–algorithm, and prompt–algorithm combinations, as well as trend analyses across versions.

This automated processing step is essential because manually inspecting every output file would not be feasible as the number of models, prompts, algorithms, and versions increases. The Excel file centralizes the evaluation results and makes it possible to identify weak prompts, low-performing models, algorithms that are harder to verify, and improvements or regressions between iterations, being easier to, based on the collected results, new iterations of the experiment can be performed while keeping the general evaluation

structure consistent.

Table 3.1: Implementation structure of the evaluation framework

Part of the framework	Project artifact	Role in the experiment
Algorithm inputs	`algorithms/`	Stores the normalized algorithm descriptions.
Property inputs	`properties/`	Stores the properties that each model must verify.
Prompt inputs	`prompts/`	Stores the static prompt templates.
Experiment runner	Model execution python script	Reads algorithms, properties, and prompts; sends queries to models.
Raw outputs	`results/`	Stores model responses by algorithm, prompt type, and model.
Results parser	Results analysis pyhton script	Reads output files and classifies each response.
Final analysis	Excel results file	Stores scores, rankings, heatmaps, and comparisons.

3.1.1 Algorithm and Prompt Repositories Component

3.1.1.A Problem Selection

The evaluation focuses on broadcast distributed algorithms, selected because they represent core communication abstractions in distributed systems while still allowing the verification task to be controlled and comparable across models. Each algorithm is represented with its corresponding system assumptions, fault model, and correctness properties as well as other algorithm's related information. In order to avoid bias caused by differences in presentation, all algorithms were rewritten using the same structured template as represented in 3.2:

Table 3.2: Structure of the algorithm input template

Template field	**Purpose**
Identification	Specifies the algorithm name, problem type, fault model, and timing model.
System model	Defines the processes, maximum faulty processes, communication assumptions, and timing assumptions.
Correctness properties	Lists the properties that the algorithm must satisfy.
Interface/API	Defines the external operations and output events.
Roles	Describes the roles used by the algorithm, such as broadcaster, receivers, senders, or witnesses.
Local state per process	Specifies the constants and variables stored by each process.
Message types and validation rules	Defines message formats and rules for accepting or counting messages.
Initialization	Provides the initial values of local variables.
Protocol pseudocode	Describes the actions executed on external invocation and message reception.
Decision/delivery rule	States the exact condition under which a process decides or delivers.
Quorum/key structural argument	Summarizes the propagation, quorum, or threshold idea behind correctness.

The selected algorithms are presented in 3.3. These include Best-Effort Broadcast, Reliable Broadcast, Terminating Reliable Broadcast, Echo Broadcast, and Fault-Tolerant Reliable Broadcast variants. Together, these algorithms cover different correctness properties and failure assumptions, including crash, omission, Byzantine, and no-failure settings. Fault-Tolerant Reliable Broadcast algorithms generated through reinforcement learning were also included because they are less standard than classical broadcast algorithms, which enables testing models' ability to reason about the provided algorithm description instead of relying only on well-known examples from training data.

Table 3.3: Algorithms Included in the Evaluation

Algorithm	Fault Model	System Model
Best-Effort Broadcast	Crash	Asynchronous
Reliable Broadcast	Crash, Omission	Asynchronous
Echo Broadcast	Byzantine	Asynchronous
Terminating Reliable Broadcast (TRB)	Crash, Omission	Synchronous
Most Efficient RBcast Algorithm for No-Failure Experiment	No failures	Asynchronous
Most Efficient RBcast Algorithm for Crash-Failure Experiment	Crash	Asynchronous
New Byzantine-Tolerant Algorithm Generated by the RB-Learner	Byzantine	Asynchronous

Consensus algorithms were initially also considered to be part of the algorithm set, however they were later excluded from the evaluation due to their verification usually requiring additional assumptions and a more complex reasoning process. Additionally, consensus algorithms often rely on broadcast abstractions to exchange information between processes, as a result, verifying consensus would require reasoning about both the consensus protocol and the broadcast abstraction used by it, which would move the evaluation away from the main goal of this work given the complexity added from this kind of protocols.

The algorithms considered in this evaluation use message passing as their communication mechanism. Depending on the algorithm, different failure assumptions are considered, as presented in 3.3 including crash failures, omission failures, Byzantine failures, and no-failure executions. These assumptions are explicitly included in the algorithm descriptions provided to the models, allowing the evaluation to test whether the models can distinguish between different fault models and adapt their reasoning accordingly.

Most of the selected algorithms operate under asynchronous assumptions, where processes do not share a global clock and there are no fixed bounds on message delays or process execution speeds. In these cases, a model answer can reveal to be weaker if it relies on timeouts, bounded delays, or synchrony assumptions that are not part of the system model. The only synchronous case included in the evaluation is Terminating Reliable Broadcast, where timing assumptions are part of the algorithm model.

3.1.1.B Prompt Design

The second component of the system is the prompt template repository, where five prompt engineering strategies are considered:

- Zero-shot prompting - evaluates the model without examples;
- Few-shot prompting - provides examples of the expected verification format;

- Chain-of-thought prompting - encourages step-by-step reasoning before producing the final verdict;
- Strict-format prompting - constrains the answer format to make comparison easier;
- Role-based prompting - assigns the model the role of a distributed systems verifier.

These prompt types were selected to evaluate whether the formulation of the prompt influences the ability of the models to identify correctness properties, reason about failures, and detect violations in distributed algorithms. Each template follows the same general goal, but presents the task to the model in a different way, including both a space for the algorithm and the properties to be verified.

3.1.2 Model Interface component

To evaluate how model scale influences the ability of language models to verify distributed algorithms, the experiments include both Large Language Models (LLMs) and Small Language Models (SLMs). The LLMs are used as upper-bound references, since they represent strong general-purpose reasoning systems with demonstrated performance in code reasoning, long-context understanding, and structured problem solving [**?**, **?**]. The SLMs are included to study whether smaller and more efficient models can still analyze pseudocode, reason about invariants, and explain correctness or errors in distributed algorithms.

The selected LLMs are GPT-4.1, Claude 3.5 Sonnet, Gemini 1.5 Pro, DeepSeek-R1, and Llama 3 70B. The selected SLMs are Yi-Coder 9B Chat, DeepSeek-Coder 6.7B, OpenCode Interpreter 6.7B, Phi-2, and Qwen2.5-Coder 7B. Together, these models provide a diverse comparison across model sizes, architectures, and reasoning capabilities, allowing the evaluation to assess whether large-scale reasoning is necessary for distributed algorithm verification or whether smaller models can achieve competitive results.

Included in all prompts is a required evaluation output for the models, intentionally restricted to make the model answers easier and faster to compare. Each response must contain at most one line per correctness property, following one of the two formats:

- `<property> - [satisfied]`
- `<property> - [non satisfied]: <reason>`

A reason must only be included when the property is classified as non-satisfied. The reason must be concise, with at most 20 words. The output must not include additional reasoning, explanations, or extra text. This restriction reduces the possibility of unnecessary or hallucinated content and allows the evaluation to focus on whether the model can correctly identify if each property is satisfied.

3.1.3 Evaluation Layer Component

The evaluation compares each model output with the expected correctness result for each algorithm property. Since the selected algorithms are assumed to be correct according to their source material, a satisfied

verdict is considered correct when it matches the expected property. When a model reports that a property is not satisfied, the provided reason or counterexample is manually inspected and, if the explanation is invalid, the result is recorded as a model verification error, if the explanation reveals that the algorithm description or prompt was ambiguous, the input is refined and the experiment may be repeated. To make this process consistent across all stored outputs, the results analysis script assigns one of four classifications to each response, as shown in Table 3.4. These classifications are later used to compute success rates and compare models, prompt strategies, algorithms, and experimental versions.

Table 3.4: Classification rules used by the results analysis script

Classification	**Condition checked by the script**
Correct	The response follows a recognized format and all expected properties are marked as satisfied.
Wrong	The response is structured, but at least one expected property is marked as non-satisfied, false, violated, or not met.
Incorrect output format	The response does not follow the expected structure, mentions fewer than half of the expected properties, uses wrong property names, or is mostly unstructured prose.
Error	The response file is empty or contains an execution problem, such as a timeout, rate limit, connection error, or unusable JSON-stream response.

3.2 Summary

This chapter presented the implementation of the evaluation framework used to assess the ability of language models to verify distributed algorithms. First, the system architecture was described, including the algorithm repository, prompt template repository, model interface, and response storage and, the data flow of the framework was then explained, showing how static algorithm descriptions and prompt templates are combined, submitted to the selected models, stored, and later evaluated.

The chapter also defined the experimental setup, the evaluation focuses on broadcast-based distributed algorithms covering different fault models and system assumptions, including crash, omission, Byzantine, no-failure, asynchronous, and synchronous settings and the selected prompt engineering strategies and the language models used in the experiments were also presented, allowing the evaluation to compare both model scale and prompt design.

Finally, the chapter described the experimental framework and implementation details. All algorithms were rewritten using a common input template to reduce presentation bias, and the model outputs were restricted to a simple property-based format to make comparison easier without accounting for reasoning capabilities, as this would increase a lot the complexity of the analysis and is not the goal of this work. The implementation is semi-automated, having a script that generates all algorithm and prompt combinations,

queries the models, and stores the responses, while the final assessment of the answers is performed manually. The next chapter presents and discusses the results obtained from this evaluation.

3.3 next

During the experiments, one model consistently produced very low-quality results. After five iterations across the set of algorithms and prompt types, this model achieved only a 4% success rate and, since it consumed a significant amount of execution time while producing outputs that did not contribute meaningfully to the evaluation, it was removed from the sample of models. This decision reduced resource usage and improved the efficiency of subsequent iterations.

diversificamos properties e algoritmos porque por vezes os modelos nao conseguiam distinguir o que eram as propriedades a verificar, e verificavam tambem outros tipos de coisas other kind of verifications beyond properties, como liveness, message complexity, message size, etc

4

Evaluation

Contents

4.1 Evaluation Procedure 30

4.2 Robustness Tests with Modified Algorithm Descriptions 31

4.1 Evaluation Procedure

The evaluation phase was done iteratively, using different combinations of algorithms, correctness properties, and prompt formats, where for each experiment the model received the algorithm description with its set of properties to verify, comparing the output against the expected classification for each property. A response was considered successful when the model correctly identified whether the given property was satisfied or violated and provided a justification consistent with the algorithm specification.

During the first iterations, the prompts and inputs were progressively refined because several models did not easily identify the specific properties requested for verification and instead evaluated unrelated dimensions present in the algorithm specification, such as liveness, message complexity, message size, or general implementation efficiency. To reduce this ambiguity, the algorithm descriptions were diversified and reformulated, the prompts were made more explicit by restricting the expected output to one line per property indicating only whether it was satisfied or not (with a short reason permitted only for non-satisfied properties), and a separate properties folder was created containing, for each algorithm, solely the comma-separated list of properties to verify, making the verification target unambiguous.

In addition to the normal iterative refinements due to the observed results, the experiments included repeated executions of the same prompt–algorithm combinations, in order to assess whether model responses were deterministic or probabilistic. The selected tests were executed in a loop of four runs using three SLMs: Qwen2.5-Coder-7B, DeepSeek-Coder-6.7B, and Yi-Coder-9B-Chat. These models chosen for their accessibility through local API requests and for being free of costs. Qwen2.5-Coder-7B proved to be fully deterministic, producing identical classifications across all four runs for every prompt combination, which established it as a reliable representative in single-run evaluations. DeepSeek-Coder-6.7B, in contrast, was highly probabilistic: out of 16 test cases only 43.8% produced consistent output across runs, with the inconsistent results varying between wrong verdicts, incorrect output formats, and connection errors, demonstrating that a single execution cannot reliably represent this model's capability. Yi-Coder-9B-Chat was the most unstable, achieving consistency in only 25% of cases, with the remaining cases producing different classifications across executions. These findings confirmed that consistency is an essential factor when interpreting model performance and that single-run evaluations are only representative for deterministic models such as Qwen2.5-Coder-7B.

Models and prompt configurations that consistently underperformed were progressively removed from the evaluation. Phi-2 was eliminated after five iterations because it achieved only a 2.6% cumulative success rate while consuming disproportionate execution time, and the few-shot prompt configuration was removed because its success rate remained below 50% across all tested models. DeepSeek-Coder-6.7B and Yi-Coder-9B-Chat were removed after version 13, having stagnated at cumulative success rates of 49.0% and 55.2% respectively. GPT-4o achieved 100% accuracy in the cases where it returned a response, but was removed because cost and rate-limit constraints prevented systematic testing. Table **??** summarizes

these elimination decisions alongside the final cumulative success rates for each model, where errors, timeouts, and rate-limit failures are excluded from the calculation since they represent failures to obtain a usable response rather than incorrect verification results.

This progressive filtering left Qwen2.5-Coder-7B as the sole model maintained throughout all versions, reaching a final cumulative success rate of 94.5% across 346 valid tests. Table **??** provides the detailed per-version and cumulative success rates, showing how each model's performance evolved across iterations.

4.2 Robustness Tests with Modified Algorithm Descriptions

To evaluate whether models were reasoning about the protocol logic itself or merely relying on superficial cues such as the algorithm name or the declared fault model, robustness tests were conducted using three modified versions of the Byzantine reliable broadcast algorithm, where the pseudocode remained identical to the correct original in all three cases but different aspects of the surrounding specification were altered.

In the first variant (-n), the algorithm name was changed to "Best Effort Broadcast" while the Byzantine reliable broadcast protocol remained unchanged; in the second variant (-o), the fault model label was changed from Byzantine to omission faults without altering the protocol logic; and in the third variant (-p), the set of properties to verify was modified by replacing RB-Integrity with RB-Termination. Since the pseudocode is identical across all variants and implements a correct Byzantine reliable broadcast, the properties RB-Validity, RB-Agreement, and RB-Integrity are genuinely satisfied regardless of the declared name or fault model — meaning that the correct answer for variants -n and -o is to mark all properties as satisfied. Only the -p variant constitutes a genuine negative test, because RB-Termination (all correct processes eventually deliver) is not guaranteed by this algorithm in an asynchronous Byzantine model: if the broadcaster is Byzantine and never sends the initial message, no correct process can ever reach the delivery threshold.

These tests were initially performed with Qwen2.5-Coder-7B, since it was the best-performing model at that stage of the evaluation. The model achieved 97% accuracy on the -n variant (29/30 valid tests) and 100% on the -o variant (28/28), confirming that it evaluates the protocol logic without being misled by superficial textual inconsistencies. On the -p variant, however, accuracy dropped to 82% (23/28), with all five failures corresponding to cases where RB-Termination was incorrectly marked as satisfied — indicating strong capability in verifying safety properties but measurable difficulty in reasoning about liveness properties when the algorithm description does not explicitly state the conditions under which termination may fail.

4.2.1 Introduction of Larger Models and Extended Robustness Testing

Given the limitations of the small local models and the need to determine whether higher-capacity models would exhibit superior verification capabilities, the evaluation was expanded starting from version 28 with the introduction of larger-scale models accessed through API services: LLaMA-3.3-70B (via Groq), Gemini-2.5-Flash-Lite, DeepSeek-Chat (V3), Gemini-2.5-Pro, and DeepSeek-R1. These models were evaluated both on the original correct algorithms and on the same modified variants previously tested with Qwen2.5-Coder-7B, enabling a direct comparison of verification capabilities across model scales.

Among the newly introduced models, Gemini-2.5-Flash-Lite achieved a perfect 100% success rate across all 24 valid tests on the modified variants, correctly identifying RB-Termination as non-satisfied in the -p variant while confirming all properties as satisfied in the -n and -o variants. DeepSeek-Chat (V3) achieved 91.7% overall (22/24), with its only failures occurring on the -p variant at 75% accuracy, reproducing the same pattern observed with Qwen2.5-Coder-7B. LLaMA-3.3-70B-Groq obtained 87.5% overall (28/32), with perfect scores on the -n and -o variants but only 50% on the -p variant, demonstrating that even a 70B-parameter model can fail to distinguish safety from liveness properties in half of its attempts. When evaluated on the original correct algorithms in versions 38–39, both DeepSeek-Chat and LLaMA-3.3-70B-Groq demonstrated near-perfect positive verification capability at 98.8% and 100% respectively across 80 tests each, revealing a clear asymmetry: high accuracy when confirming correctness versus significantly reduced accuracy when detecting that a liveness property is not guaranteed.

4.2.2 Motivation for Pseudocode-Level Errors

This asymmetry highlighted a fundamental limitation of the current experimental design: since the -n and -o variants do not introduce genuine logical errors (the pseudocode remains correct), and the -p variant tests property identification rather than protocol analysis, the evaluation had not yet measured whether models can detect actual violations of correctness properties arising from flawed protocol logic. This motivated the design of a subsequent set of experiments using algorithm variants with errors introduced directly into the pseudocode — altered delivery thresholds, removed propagation rules, and weakened triggering conditions — representing a definitive test of each model's capacity for formal property verification against incorrect algorithms.

5

This is the Fifth Chapter

Contents

5.1 Maecenas vitae nulla consequat 34

5.2 Proin ornare dignissim lacus 35

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Morbi commodo, ipsum sed pharetra gravida, orci magna rhoncus neque, id pulvinar odio lorem non turpis. Nullam sit amet enim. Suspendisse id velit vitae ligula volutpat condimentum. Aliquam erat volutpat. Sed quis velit. Nulla facilisi. Nulla libero. Vivamus pharetra posuere sapien. Nam consectetuer. Sed aliquam, nunc eget euismod ullamcorper, lectus nunc ullamcorper orci, fermentum bibendum enim nibh eget ipsum. Donec porttitor ligula eu dolor. Maecenas vitae nulla consequat libero cursus venenatis. Nam magna enim, accumsan eu, blandit sed, blandit a, eros.

5.1 Maecenas vitae nulla consequat

Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros Figure 5.1, facilisis vel, eleifend non, auctor dapibus, pede.

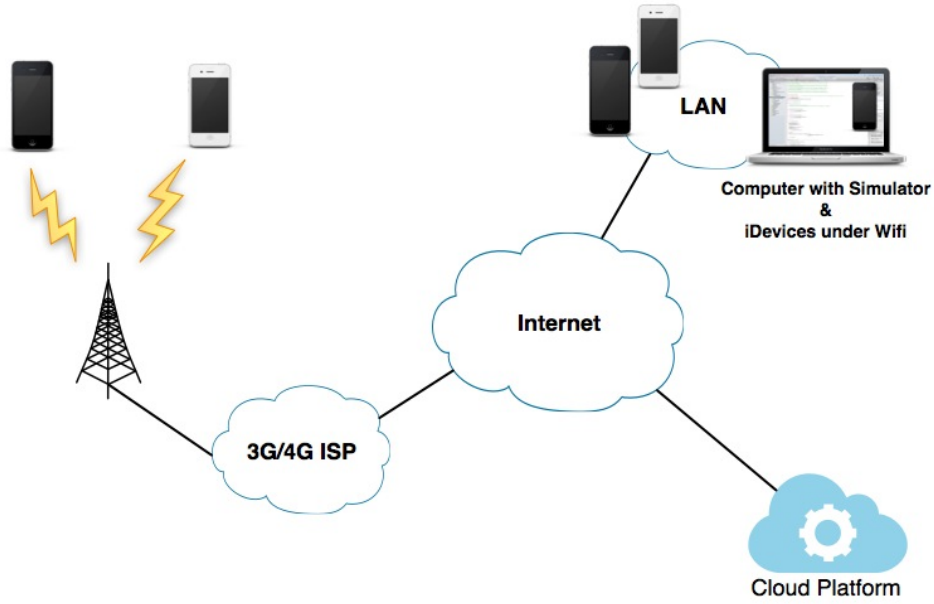


Figure 5.1: Test Environment

Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede Table 5.1 used in the tests. The Network Link Conditioner allows to force/simulate fluctuations in fixed network segments.

Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi.

Table 5.1: Network Link Conditioner Profiles

Network Profile	Bandwidth	Packets Droped	Delay
Wifi	40 mbps	0%	1 ms
3G	780 kbps	0%	100 ms
Edge	240 kbps	0%	400 ms

Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede.

5.2 Proin ornare dignissim lacus

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Mauris placerat eleifend leo. Quisque sit amet est et sapien ullamcorper pharetra. Vestibulum erat wisi, condimentum sed, commodo vitae, ornare sit amet, wisi. Aenean fermentum, elit eget tincidunt condimentum, eros ipsum rutrum orci, sagittis tempus lacus enim ac dui. Donec non enim in turpis pulvinar facilisis. Ut felis.

Et “optimistic” nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui G_j, nec ligula et lorem consequat ullamcorper p ut mauris eu mi mollis luctus j, porttitor ut, Equation (5.1), uctus posuere justo:

N_j Is the number of times peer j has been optimistically unchoked.

n_j Among the N_j unchokes, the number of times that peer j responded with unchoke or supplied segments to peer p.

$C_{r[j]}$ The cooperation ratio of peer j. If peer j never supplied peer p, the information of $C_{r[j]}$ may not be available.

$C_{r(max)}$ The maximum cooperation ratio of peer p's neighbors, i.e., $C_{r(max)} = max(C_r)$.

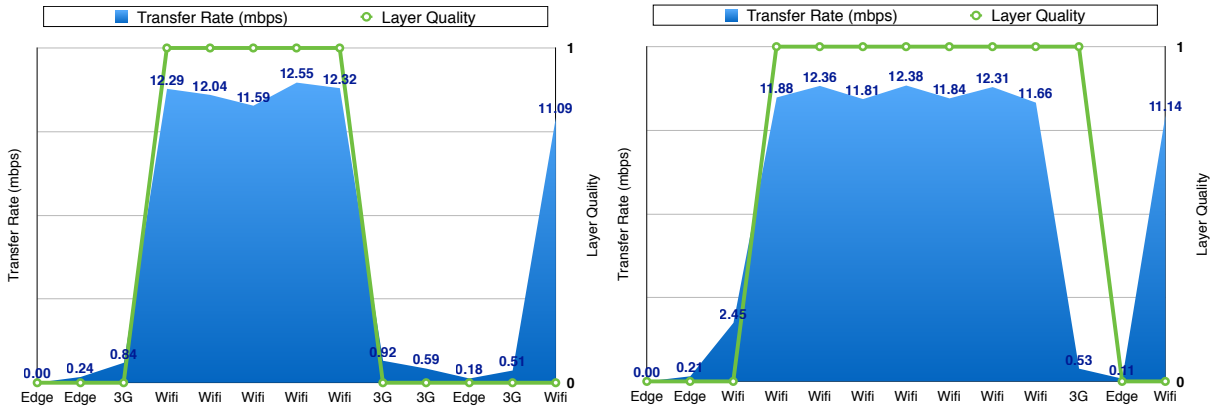
$$G_j = \begin{cases} \dfrac{n_j C_{r[j]}}{N_j} & \text{if } n_j > 0 \\ \dfrac{C_{r(max)}}{N_j + 1} & \text{if } n_j = 0 \end{cases} \tag{5.1}$$

Cursus $C_{r(max)}$ conubia nostra, per inceptos hymenaeos j gadipiscing mollis massa $N_j = 0$, unc ut dui eget nulla venenatis aliquet $G_j = C_{r(max)}$.

Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat

ullamcorper. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Phasellus eget nisl ut elit porta ullamcorper. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa.

Both Figures 5.2(a) and 5.2(b) Phasellus eget nisl ut elit porta “perfect” tincidunt. Class aptent taciti sociosqu ad litora torquent per conubia nostra.



(a) Adaptation System Test 4

(b) Adaptation System Test 5

Figure 5.2: Adaptation System Behavior. a) Test 4. b) Test 5

Cras sed ante. Phasellus in massa. Curabitur dolor eros, gravida et, hendrerit ac, cursus non, massa. Aliquam lorem. In hac habitasse platea dictumst. Cras eu mauris. Quisque lacus. Donec ipsum. Nullam vitae sem at nunc pharetra ultricies. Vivamus elit eros, ullamcorper a, adipiscing sit amet, porttitor ut, nibh. Maecenas adipiscing mollis massa. Nunc ut dui eget nulla venenatis aliquet. Sed luctus posuere justo. Cras vehicula varius turpis. Vivamus eros metus, tristique sit amet, molestie dignissim, malesuada et, urna.

6

Conclusion

Contents

6.1 Conclusions 38

6.2 System Limitations and Future Work 39

Pellentesque vel dui sed orci faucibus iaculis. Suspendisse dictum magna id purus tincidunt rutrum. Nulla congue. Vivamus sit amet lorem posuere dui vulputate ornare. Phasellus mattis sollicitudin ligula. Duis dignissim felis et urna. Integer adipiscing congue metus.

Rui Cruz
You should always start a Chapter with an introductory text

6.1 Conclusions

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Morbi commodo, ipsum sed pharetra gravida, orci magna rhoncus neque, id pulvinar odio lorem non turpis. Nullam sit amet enim. Suspendisse id velit vitae ligula volutpat condimentum. Aliquam erat volutpat. Sed quis velit. Nulla facilisi. Nulla libero. Vivamus pharetra posuere sapien. Nam consectetuer. Sed aliquam, nunc eget euismod ullamcorper, lectus nunc ullamcorper orci, fermentum bibendum enim nibh eget ipsum. Donec porttitor ligula eu dolor. Maecenas vitae nulla consequat libero cursus venenatis. Nam magna enim, accumsan eu, blandit sed, blandit a, eros.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetuer erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis. Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus. Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

Proin at eros non eros adipiscing mollis. Donec semper turpis sed diam. Sed consequat ligula nec tortor. Integer eget sem. Ut vitae enim eu est vehicula gravida. Morbi ipsum ipsum, porta nec, tempor id, auctor vitae, purus. Pellentesque neque. Nulla luctus erat vitae libero. Integer nec enim. Phasellus aliquam enim et tortor. Quisque aliquet, quam elementum condimentum feugiat, tellus odio consectetuer wisi, vel nonummy sem neque in elit. Curabitur eleifend wisi iaculis ipsum. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. In non velit non ligula laoreet ultrices. Praesent ultricies facilisis nisl. Vivamus luctus elit sit amet mi. Phasellus pellentesque, erat eget elementum volutpat, dolor nisl porta neque, vitae sodales ipsum nibh in ligula. Maecenas mattis pulvinar diam. Curabitur sed leo.

Nulla facilisi. In vel sem. Morbi id urna in diam dignissim feugiat. Proin molestie tortor eu velit. Aliquam erat volutpat. Nullam ultrices, diam tempus vulputate egestas, eros pede varius leo, sed imperdiet lectus est ornare odio. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Proin consectetuer velit in dui. Phasellus wisi purus, interdum vitae, rutrum accumsan, viverra in, velit. Sed enim risus, congue non, tristique in, commodo eu, metus. Aenean tortor mi, imperdiet id, gravida eu, posuere eu, felis. Mauris sollicitudin, turpis in hendrerit sodales, lectus ipsum pellentesque ligula, sit amet scelerisque urna nibh ut

arcu. Aliquam in lacus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nulla placerat aliquam wisi. Mauris viverra odio. Quisque fermentum pulvinar odio. Proin posuere est vitae ligula. Etiam euismod. Cras a eros.

Nunc auctor bibendum eros. Maecenas porta accumsan mauris. Etiam enim enim, elementum sed, bibendum quis, rhoncus non, metus. Fusce neque dolor, adipiscing sed, consectetuer et, lacinia sit amet, quam.

6.2 System Limitations and Future Work

Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede.

Bibliography

A

Code of Project

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Listing A.1: Example of a XML file.

```
<?xml version="1.0" encoding="UTF-8"?>
<StreamInfo version="2.0">
    <Clip duration="PT01M0.00S">
        <BaseURL>videos/</BaseURL>
        <Description>svc_1</Description>
        <Representation mimeType="video/SVC" codecs="svc" frameRate="30.00" bandwidth="401.90"
            width="176" height="144" id="L0">
            <BaseURL>svc_1/</BaseURL>
            <SegmentInfo from="0" to="11" duration="PT5.00S">
                <BaseURL>svc_1-L0-</BaseURL>
            </SegmentInfo>
        </Representation>
        <Representation mimeType="video/SVC" codecs="svc" frameRate="30.00" bandwidth="1322.60"
            width="352" height="288" id="L1">
            <BaseURL>svc_1/</BaseURL>
            <SegmentInfo from="0" to="11" duration="PT5.00S">
```

```
                <BaseURL>svc_1-L1-</BaseURL>
            </SegmentInfo>
        </Representation>
    </Clip>
</StreamInfo>
```

Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa. Duis dignissim euismod quam.

Listing A.2: Assembler Main Code.

```
; ********************************************************************
; * Constantes
; ********************************************************************

ON    EQU 1 ; contagem ligada
OFF   EQU 0 ; contagem desligada
INPUT EQU 8000H ; endereco do porto de entrada
          ;(bit 0 = RTC; bit 1 = botao)
OUTPUT  EQU 8000H ; endereco do porto de saida.


; ********************************************************************
; * Stack
; ********************************************************************

PLACE   1000H
pilha:    TABLE 100H  ; espaco reservado para a pilha
fim_pilha:

; ********************************************************************

PLACE   2000H

; Tabela de vectores de interrupcao

tab:    WORD  rot0

; ********************************************************************
; * Programa Principal
; ********************************************************************

PLACE   0

inicio:
  MOV BTE, tab    ; incializa BTE
  MOV R9, INPUT   ; endereco do porto de entrada
  MOV R10, OUTPUT   ; endereco do porto de saida
  MOV SP, fim_pilha
  MOV R5, 1     ; inicializa estado do processo P1
  MOV R6, 1     ; inicializa estado do processo P2
  MOV R4, OFF     ; inicializa controle de RTC
  MOV R8, 0     ; inicializa contador
  MOV R7, OFF     ; inicialmente nao permite contagem
  EI0         ; permite interrupcoes tipo 0
  EI          ; activa interrupcoes

ciclo:
  CALL  P1      ; invoca processo P1
  CALL  P2      ; invoca processo P2
  JMP   ciclo   ; repete ciclo

; ********************************************************************
;* ROTINAS
; ********************************************************************

P1:
  CMP R5, 1   ; se estado = 1
  JZ  P1_1
  CMP R5, 2   ; se estado = 2
```

```
60    JZ  P1_2
61  sai_P1:
62    RET       ; sai do processo.
63
64
65  P1_1:
66    MOVB  R0, [R9]  ; ler porto de entrada
67    BIT R0, 1
68    JZ  sai_P1    ; se botao nao carregado, sai do processo
69    MOV R7, ON    ; permite contagem do display
70    MOV R5, 2   ; passa ao estado 2 do P1
71    JMP sai_P1
72
73  P1_2:
74    MOVB  R0, [R9]  ; ler porto de entrada
75    BIT R0, 1
76    JNZ sai_P1    ; se botao continua carregado, sai do processo
77    MOV R7, OFF   ; caso contrario, desliga contagem do display
78    MOV R5, 1   ; passa ao estado 1 do P1
79    JMP sai_P1
```

Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Phasellus eget nisl ut elit porta ullamcorper. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos.

Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Phasellus eget nisl ut elit porta ullamcorper. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa. Duis dignissim euismod quam. Nullam euismod metus ut orci. Vestibulum erat libero, scelerisque et, porttitor et, varius a, leo.

Listing A.3: HTML with CSS Code

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Listings Style Test</title>
5      <meta charset="UTF-8">
6      <style>
7        /* CSS Test */
8        * {
9          padding: 0;
10         border: 0;
11         margin: 0;
12       }
13     </style>
14     <link rel="stylesheet" href="css/style.css" />
15   </head>
16   <header> hey </header>
17   <article> this is a article </article>
18   <body>
19     <!-- Paragraphs are fine -->
```

```
    <div id="box">
      <p>
        Hello World
      </p>
      <p>Hello World</p>
      <p id="test">Hello World</p>
      <p></p>
    </div>
    <div>Test</div>
    <!-- HTML script is not consistent -->
    <script src="js/benchmark.js"></script>
    <script>
      function createSquare(x, y) {
        // This is a comment.
        var square = document.createElement('div');
        square.style.width = square.style.height = '50px';
        square.style.backgroundColor = 'blue';

        /*
         * This is another comment.
         */
        square.style.position = 'absolute';
        square.style.left = x + 'px';
        square.style.top = y + 'px';

        var body = document.getElementsByTagName('body')[0];
        body.appendChild(square);
      };

      // Please take a look at +=
      window.addEventListener('mousedown', function(event) {
        // German umlaut test: Berührungspunkt ermitteln
        var x = event.touches[0].pageX;
        var y = event.touches[0].pageY;
        var lookAtThis += 1;
      });
    </script>
  </body>
```

```
</html>
```

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Listing A.4: HTML CSS Javascript Code

```
@media only screen and (min-width: 768px) and (max-width: 991px) {

  #main {
    width: 712px;
    padding: 100px 28px 120px;
  }

  /* .mono {
    font-size: 90%;
  } */

  .cssbtn a {
    margin-top: 10px;
    margin-bottom: 10px;
    width: 60px;
    height: 60px;
    font-size: 28px;
    line-height: 62px;
  }
```

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Listing A.5: PYTHON Code

```
1 class TelgramRequestHandler(object):
2     def handle(self):
3         addr = self.client_address[0]          # Client IP-adress
4         telgram = self.request.recv(1024)      # Recieve telgram
5         print "From: %s, Received: %s" % (addr, telgram)
6         return
```

B

A Large Table

Aliquam et nisl vel ligula consectetuer suscipit. Morbi euismod enim eget neque. Donec sagittis massa. Vestibulum quis augue sit amet ipsum laoreet pretium. Nulla facilisi. Duis tincidunt, felis et luctus placerat, ipsum libero vestibulum sem, vitae elementum wisi ipsum a metus. Nulla a enim sed dui hendrerit lobortis. Donec lacinia vulputate magna. Vivamus suscipit lectus at quam. In lectus est, viverra a, ultricies ut, pulvinar vitae, tellus. Donec et lectus et sem rutrum sodales. Morbi cursus. Aliquam a odio. Sed tortor velit, convallis eget, porta interdum, convallis sed, tortor. Phasellus ac libero a lorem auctor mattis. Lorem ipsum dolor sit amet, consectetuer adipiscing elit.

Nunc auctor bibendum eros. Maecenas porta accumsan mauris. Etiam enim enim, elementum sed, bibendum quis, rhoncus non, metus. Fusce neque dolor, adipiscing sed, consectetuer et, lacinia sit amet, quam. Suspendisse wisi quam, consectetuer in, blandit sed, suscipit eu, eros. Etiam ligula enim, tempor ut, blandit nec, mollis eu, lectus. Nam cursus. Vivamus iaculis. Aenean risus purus, pharetra in, blandit quis, gravida a, turpis. Donec nisl. Aenean eget mi. Fusce mattis est id diam. Phasellus faucibus interdum sapien. Duis quis nunc. Sed enim. Nunc auctor bibendum eros. Maecenas porta accumsan mauris. Etiam enim enim, elementum sed, bibendum quis, rhoncus non, metus. Fusce neque dolor, adipiscing sed, consectetuer et, lacinia sit amet, quam.

As Table B.1 shows, the data can be inserted from a file, in the case of a somehow complex structure. Notice the Table footnotes.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Morbi commodo, ipsum sed pharetra gravida, orci magna rhoncus neque, id pulvinar odio lorem non turpis. Nullam sit amet enim. Suspendisse id velit vitae ligula volutpat condimentum. Aliquam erat volutpat. Sed quis velit. Nulla facilisi. Nulla libero. Vivamus

Table B.1: Example table

Benchmark: ANN	**#Layers** (1)	**#Nets** (2)	#Nodes* (3) = 8 · (1) · (2)	Critical path (4) = 4 · (1)	**Latency (T_{iter})** (5)
A1	**3–1501**	1	**24–12008**	**12–6004**	4
A2	501	1	4008	2004	**2–2000**
A3	10	**2–1024**	**160–81920**	40	60†
A4	10	50	4000	40	**80–1200**

Benchmark: FFT	**FFT size‡** (1)	*#Inputs* (2) = $2^{(1)}$	*#Nodes** (3) = 10 · (1) · (2)	*Critical path* (4) = 4 · (1)	**Latency (T_{iter})** (5)
F1	**1–10**	2–1024	**20–102400**	4–40	6–60†
F2	**5**	32	1600	20	**40 – 1500**

Benchmark: Random networks	**#Types** (1)	**#Nodes** (2)	*#Networks* (3)	*Critical path* (4)	**Latency (T_{iter})** (5)
R1	3	10–2000	500	*variable*	(4)
R2	3	50	500	*variable*	(4) × [1; · · · ; 20]

* *Excluding constant nodes.*
† *Value kept proportional to the critical path:* $(5) = (4) * 1.5$.
‡ *A size of x corresponds to a 2^x point FFT.*
Values in bold indicate the parameter being varied.

pharetra posuere sapien. Nam consectetuer. Sed aliquam, nunc eget euismod ullamcorper, lectus nunc ullamcorper orci, fermentum bibendum enim nibh eget ipsum. Donec porttitor ligula eu dolor. Maecenas vitae nulla consequat libero cursus venenatis. Nam magna enim, accumsan eu, blandit sed, blandit a, eros.

And now an example (Table B.2) of a table that extends to more than one page. Notice the repetition of the Caption (with indication that is continued) and of the Header, as well as the continuation text at the bottom.

Table B.2: Example of a very long table spreading in several pages

Time (s)	**Triple chosen**	**Other feasible triples**
0	(1, 11, 13725)	(1, 12, 10980), (1, 13, 8235), (2, 2, 0), (3, 1, 0)
2745	(1, 12, 10980)	(1, 13, 8235), (2, 2, 0), (2, 3, 0), (3, 1, 0)
5490	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
8235	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
10980	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
13725	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
16470	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
19215	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
21960	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
24705	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
27450	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
30195	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
32940	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)

Continued on next page...

Table B.2: (...continued from previous page)

Time (s)	Triple chosen	Other feasible triples
35685	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
38430	(1, 13, 10980)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
41175	(1, 12, 13725)	(1, 13, 10980), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
43920	(1, 13, 10980)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
46665	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
49410	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
52155	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
54900	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
57645	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
60390	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
63135	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
65880	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
68625	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
71370	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
74115	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
76860	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
79605	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
82350	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
85095	(1, 12, 13725)	(1, 13, 10980), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
87840	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
90585	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
93330	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
96075	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
98820	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
101565	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
104310	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
107055	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
109800	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
112545	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
115290	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
118035	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
120780	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
123525	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
126270	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
129015	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
131760	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
134505	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
137250	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
139995	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
142740	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
145485	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
148230	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
150975	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
153720	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
156465	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
159210	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
161955	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
164700	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)

An example of a large Table that autofits the size to the page margins is illustrated in Table B.3. Please notice the text size that is shrunken in order fot the table to adjust to the page:

Table B.3: Sample Table.

URL	First Time Visit	Last Time Visit	URL Counts	Value	Reference
https://web.facebook.com/	1521241972	1522351859	177	56640	[facebook-2021]
http://localhost/phpmyadmin/	1518413861	1522075694	24	39312	database-management
https://mail.google.com/mail/u/	1516596003	1522352010	36	33264	Google-Gmail-2021
https://github.com/shawon100	1517215489	1522352266	37	27528	Code-Repository
https://www.youtube.com/	1517229227	1521978502	24	14792	Youtube-video-2021